

Lab Work 09

Topic	Using AI for refactoring code and generating/improving unit tests.
Objective	Apply an AI assistant to improve existing code structure and ensure code quality through automated testing.
AI Copilot Usage	Allowed and encouraged

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.

Submission

- Submit updated ***school_simulator.py***, the new/updated ***test_school_system.py***, and your prompt history for this session.

Tasks

1. AI-Assisted Refactoring:

- Identify Target:** Look at the ***Unit.add_student*** method.
- Prompt:** Copy the current ***Unit.add_student*** method. Paste it into your AI assistant and ask: "Refactor this Python method. Extract the validation logic (checking type, duplication, capacity) into separate private helper methods within the ***Unit*** class (e.g., ***_is_valid_student***, ***_is_already_enrolled***, ***_has_capacity***). The main ***add_student*** method should then call these helpers for clarity."
- Evaluate:** Review the AI's suggested refactoring. Does it correctly extract the logic? Are helper methods clear? Does the main method flow logically? Integrate the refactored code. Manually test or run existing tests to ensure functionality is preserved.

2. Generating Unit Tests:

- Target:** Choose ***calculate_grade(mark)*** or the ***Student*** constructor. Let's use ***calculate_grade***.
- Prompt:** Ask your AI assistant: "Generate Python unit tests using the 'unittest' module for the following function: [paste ***calculate_grade*** function code]. Include test cases for each grade boundary (e.g., 49 vs 50, 79 vs 80), a typical value within each

grade range, and edge cases like 0, 100, -1, and 101." (Alternatively, ask for 'pytest' format).

- c) **Evaluate:** Examine the generated test code. Create a new file (e.g., *test_school_system.py*) and paste the tests in. Does it import *unittest* (or *pytest*) and your function? Are assertions correct (*assertEqual*, etc.)? Does it cover the requested cases? Run the tests (e.g., *python -m unittest test_school_system.py*).

3. AI for Test Case Ideas & Edge Cases:

- a) **Target:** Consider the *EnrollmentSystem.enroll_student_in_unit* method.
- b) **Prompt:** Ask your AI assistant: "List the most important test cases and edge cases I should write unit tests for when testing an *enroll_student_in_unit(student_id, unit_code)* method in Python. Assume it interacts with a student registry and unit data, and handles enrollment logic including capacity and potential prerequisites."
- c) **Evaluate:** Analyze the list provided (see Session 9 Standard Answers for expected items). Does it cover success, different failure types (not found, full, prerequisites), and edge conditions? Use these ideas to manually write test stubs (or prompt the AI: "Generate a pytest test stub for the 'student not found' case for the enroll method") in your test file.

4. Creative Challenge (Optional, if time permits):

- a) **Test-Driven Development (TDD) with AI:** Follow the steps outlined in the original Session 9 Creative Challenge, using prompts to your AI assistant first for the test, then for the implementation to make the test pass.